

Accessing ADLS Gen2 from Azure Databricks using Access Key and Key Vault

Using Access Key and Azure Key Vault

This project demonstrates two ways of working with files stored in Azure Data Lake Storage Gen2 from an Azure Databricks workspace. The first part uses the storage account access key directly. The second part sets up Azure Key Vault and a Databricks secret scope so credentials can be handled more securely.

1. Accessing ADLS Gen2 from Databricks using storage access key

1.1 Resources used for the project

The resource group contains the Azure Databricks workspace and the ADLS Gen2 storage account used in the exercise.

The screenshot displays the Microsoft Azure portal interface. On the left, the 'Resource Manager | Resource groups' sidebar is visible, showing a search bar and a list of resource groups: 'Akumen', 'dbrs', and 'NetworkWatcherRG'. The main pane shows the 'Overview' tab for the 'Akumen' resource group. It lists resources: 'databricks1121' (Azure Databricks workspace) and 'storacc001121' (Storage account), both located in 'Central India'. The 'Resources' section includes a table with columns: Name, Type, and Location.

Name	Type	Location
databricks1121	Azure Databricks...	Central India
storacc001121	Storage account	Central India

1.2 Preparing the storage container

The storage account contains a source container for the input files. The container is kept private and is accessed from Databricks through authenticated storage access.

The screenshot shows the Microsoft Azure portal interface. The top navigation bar includes the Microsoft Azure logo, an 'Upgrade' button, a search bar, and a user profile. The main content area is titled 'storacc001121 | Containers'. On the left, a sidebar lists various Azure services, with 'Containers' selected under 'Data storage'. The main panel displays a table of containers. The table has columns for 'Name', 'Last modified', 'Anonymous access level', 'Lease state', and 'Storage connector'. Two containers are listed: 'logs' and 'source', both created on 9/8/2026. The 'logs' container was last modified at 12:49:29 PM, and the 'source' container at 12:50:39 PM. Both have 'Private' anonymous access levels and 'Available' lease states.

Name	Last modified	Anonymous access level	Lease state	Storage connector
logs	9/8/2026, 12:49:29 PM	Private	Available	-
source	9/8/2026, 12:50:39 PM	Private	Available	-

1.3 Getting the storage account access key

The storage account access key is taken from the Access keys page. The credential itself is masked in this report.

The screenshot shows the Microsoft Azure portal interface for the 'Access keys' page of storage account 'storacc001121'. The left sidebar shows 'Access keys' selected under 'Security + networking'. The main panel provides instructions on managing access keys and displays two keys, 'key1' and 'key2'. Each key section shows the 'Last rotated' date (9/8/2026, 0 days ago), the 'Key' (masked with a red box), and the 'Connection string' (masked with a red box). Buttons for 'Copy to clipboard', 'Hide', and 'Show' are provided for each key.

Access keys authenticate your applications' requests to this storage account. Keep your keys in a secure location like Azure Key Vault, and replace them often with new keys. The two keys allow you to replace one while still using the other.

Remember to update the keys with any Azure resources and apps that use this storage account. [Learn more about managing storage account access keys](#)

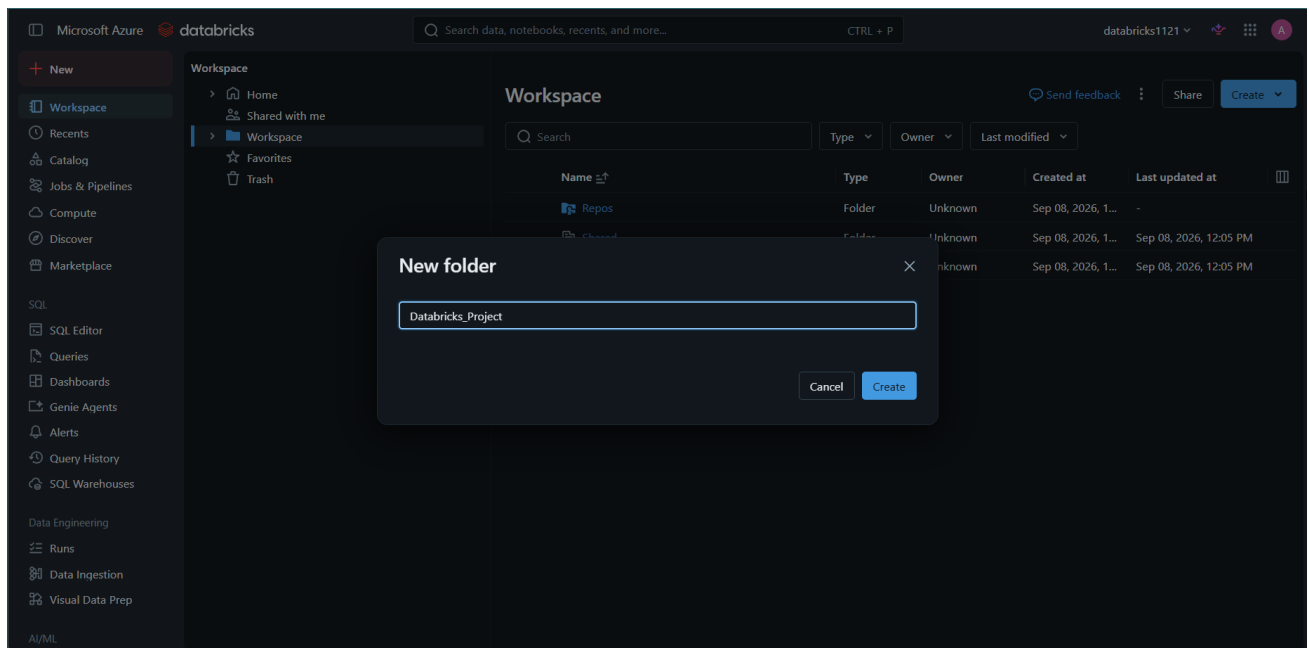
Storage account name: storacc001121

key1 Rotate key
Last rotated: 9/8/2026 (0 days ago)
Key: [Redacted] Copy to clipboard Hide
Connection string: [Redacted] Show

key2 Rotate key
Last rotated: 9/8/2026 (0 days ago)
Key: [Redacted] Show
Connection string: [Redacted] Show

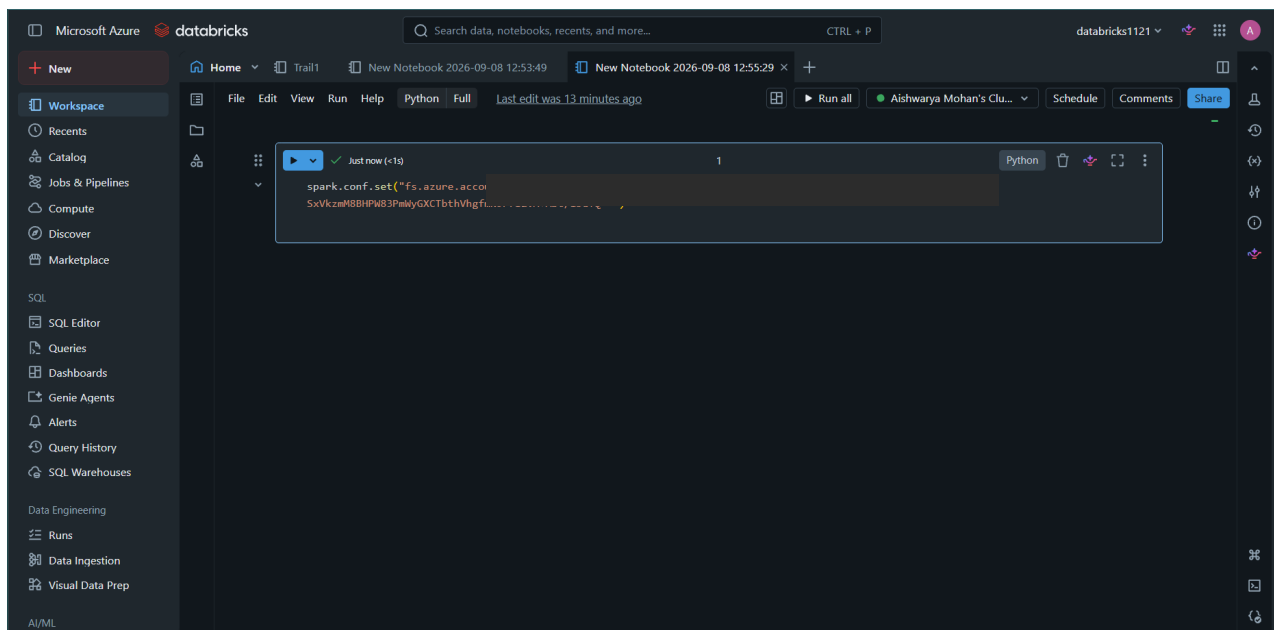
1.4 Creating a workspace folder in Databricks

A project folder is created in the Databricks workspace to keep the notebooks for this exercise together.



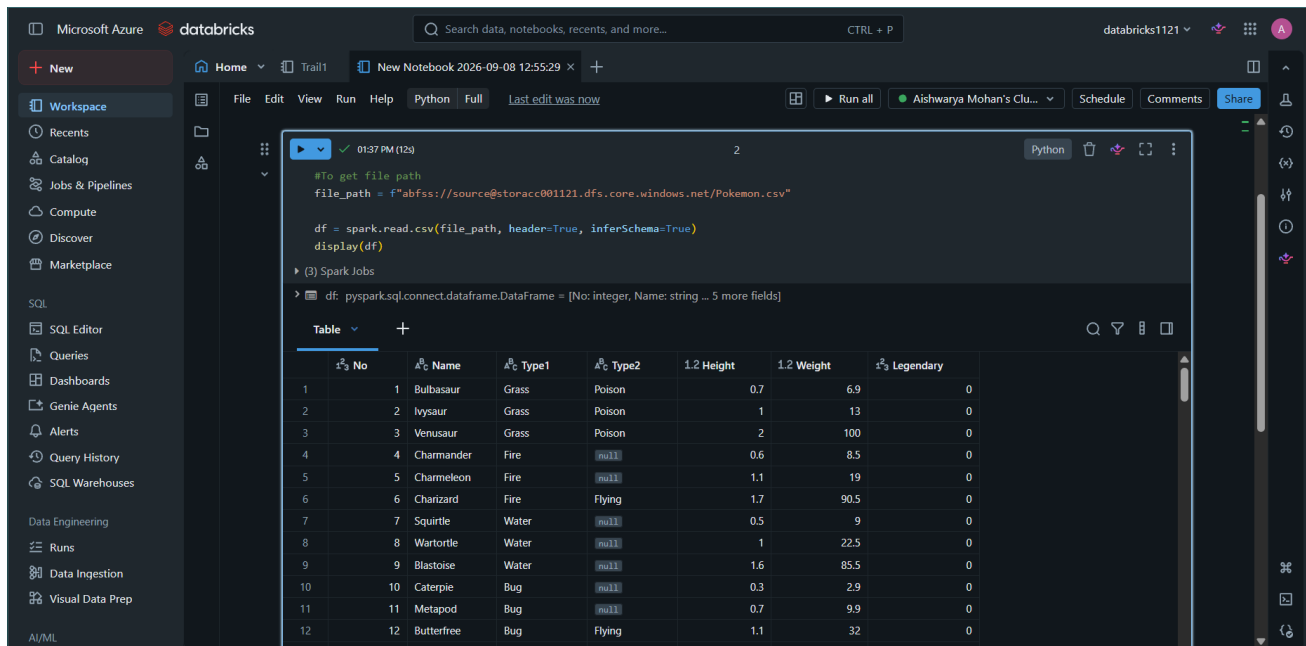
1.5 Configuring ADLS access in the notebook

The storage account key is added to the Spark configuration for the ADLS Gen2 endpoint. The key value is masked here, while the storage account endpoint remains visible.



1.6 Reading the file from ADLS Gen2

The CSV file is read through an abfss path and displayed as a Spark DataFrame. The preview confirms that the file can be reached successfully from Databricks.



The screenshot shows a Databricks notebook interface. The code in the cell reads a CSV file from an ADLS Gen2 path and displays the first 12 rows of the resulting DataFrame.

```
#To get file path
file_path = f"abfss://source@storacc001121.dfs.core.windows.net/Pokemon.csv"

df = spark.read.csv(file_path, header=True, inferSchema=True)
display(df)
```

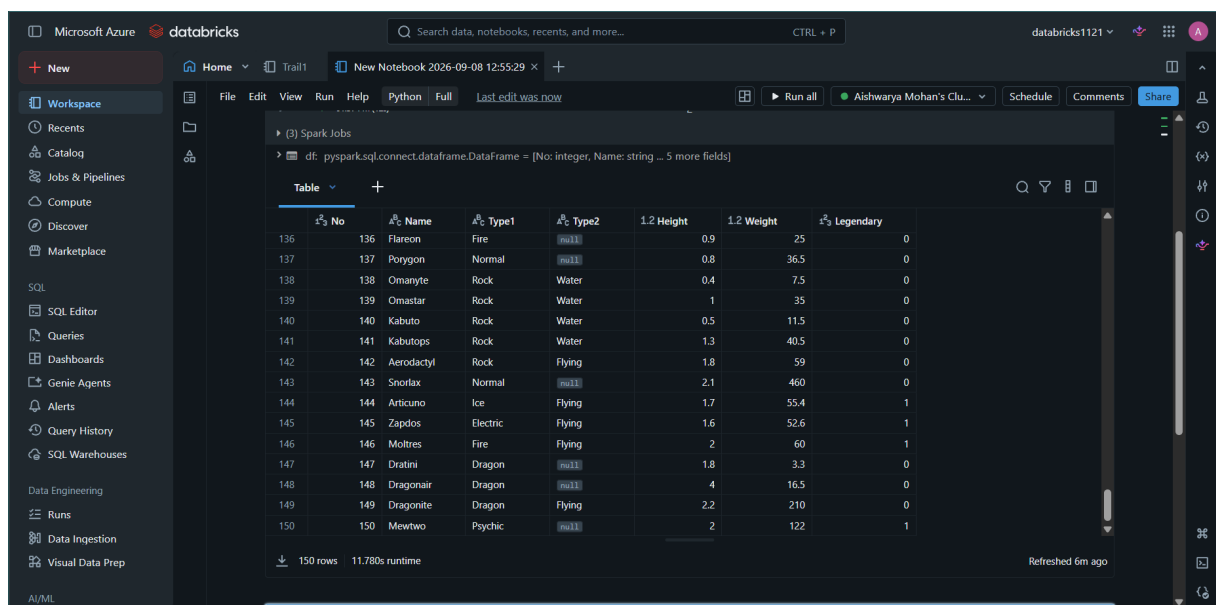
(3) Spark Jobs

df: pyspark.sql.connect.dataframe.DataFrame = [No: integer, Name: string ... 5 more fields]

No	Name	Type1	Type2	Height	Weight	Legendary
1	Bulbasaur	Grass	Poison	0.7	6.9	0
2	Ivysaur	Grass	Poison	1	13	0
3	Venusaur	Grass	Poison	2	100	0
4	Charmander	Fire	null	0.6	8.5	0
5	Charmeleon	Fire	null	1.1	19	0
6	Charizard	Fire	Flying	1.7	90.5	0
7	Squirtle	Water	null	0.5	9	0
8	Wartortle	Water	null	1	22.5	0
9	Blastoise	Water	null	1.6	85.5	0
10	Caterpie	Bug	null	0.3	2.9	0
11	Metapod	Bug	null	0.7	9.9	0
12	Butterfree	Bug	Flying	1.1	32	0

1.7 Checking the complete result

The displayed result contains 150 rows, confirming that the source file was read successfully.



The screenshot shows the Databricks notebook interface with the complete result of the CSV file read. The DataFrame contains 150 rows.

(3) Spark Jobs

df: pyspark.sql.connect.dataframe.DataFrame = [No: integer, Name: string ... 5 more fields]

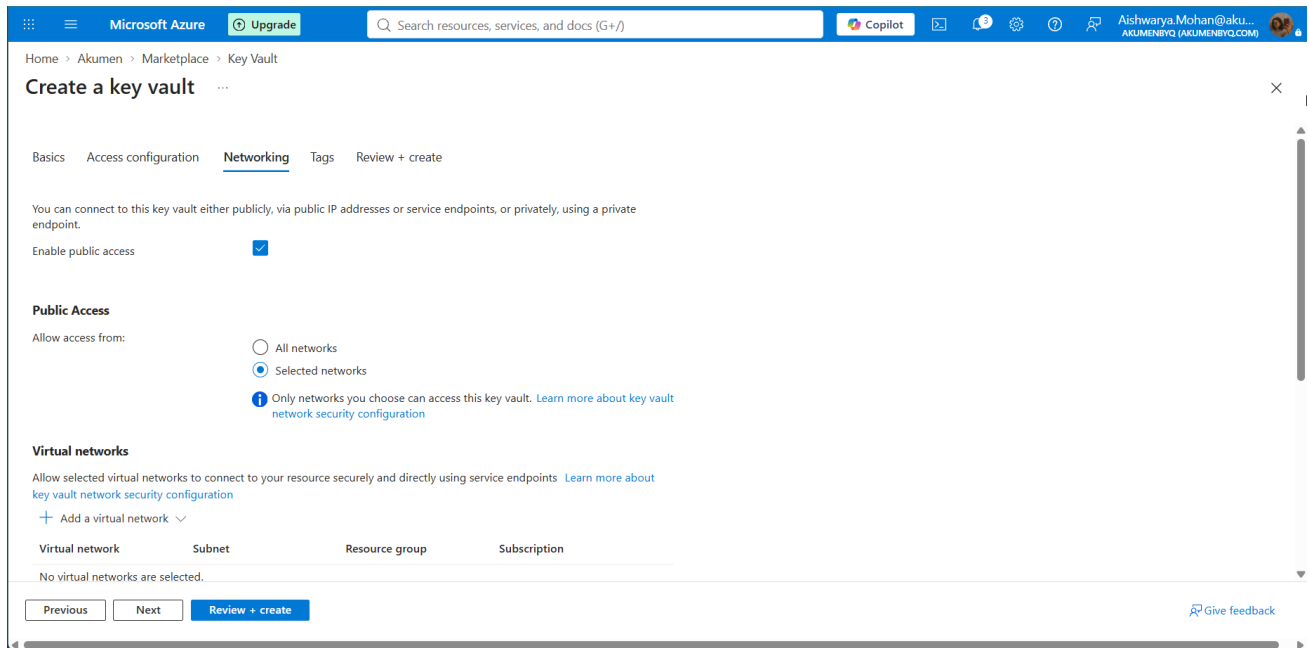
No	Name	Type1	Type2	Height	Weight	Legendary
136	Flareon	Fire	null	0.9	25	0
137	Porygon	Normal	null	0.8	36.5	0
138	Omanyte	Rock	Water	0.4	7.5	0
139	Omastar	Rock	Water	1	35	0
140	Kabuto	Rock	Water	0.5	11.5	0
141	Kabutops	Rock	Water	1.3	40.5	0
142	Aerodactyl	Rock	Flying	1.8	59	0
143	Snorlax	Normal	null	2.1	460	0
144	Articuno	Ice	Flying	1.7	55.4	1
145	Zapdos	Electric	Flying	1.6	52.6	1
146	Moltres	Fire	Flying	2	60	1
147	Dratini	Dragon	null	1.8	3.3	0
148	Dragonair	Dragon	null	4	16.5	0
149	Dragonite	Dragon	Flying	2.2	210	0
150	Mewtwo	Psychic	null	2	122	1

150 rows 11.780s runtime Refreshed 6m ago

2. Accessing secrets through Azure Key Vault

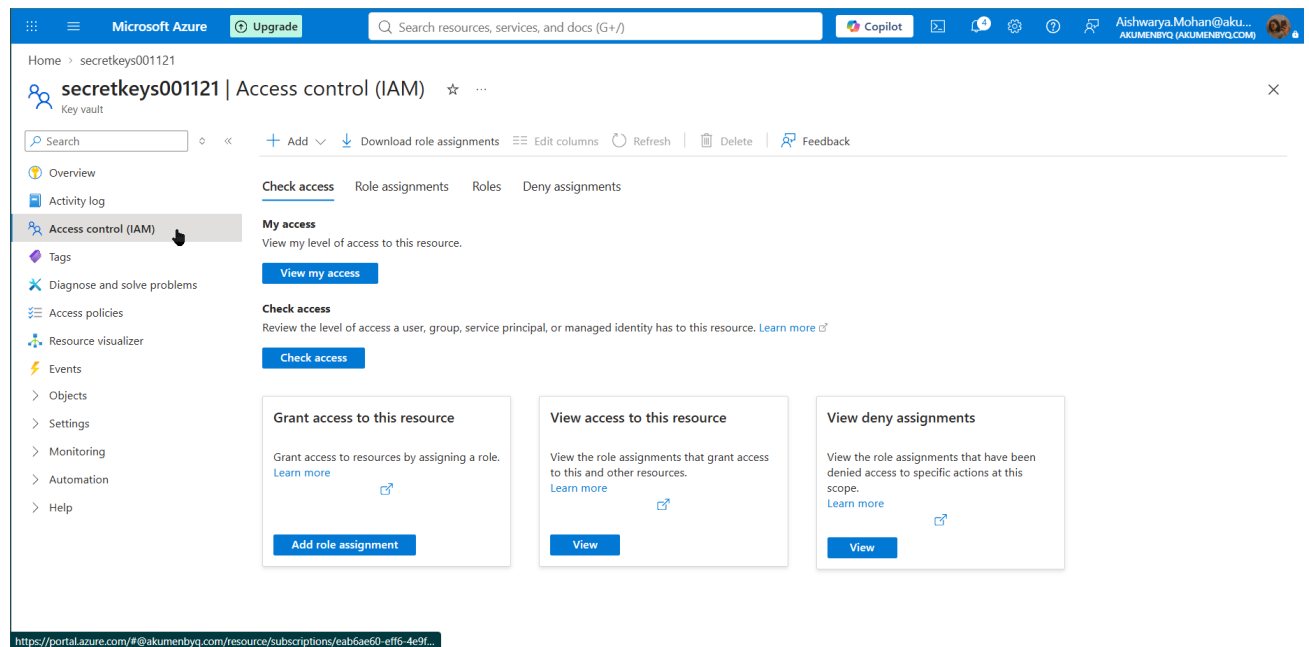
2.1 Creating and opening the Key Vault

A Key Vault is created in the same project environment. During setup, network access is configured so the vault can be reached while the integration is being tested.



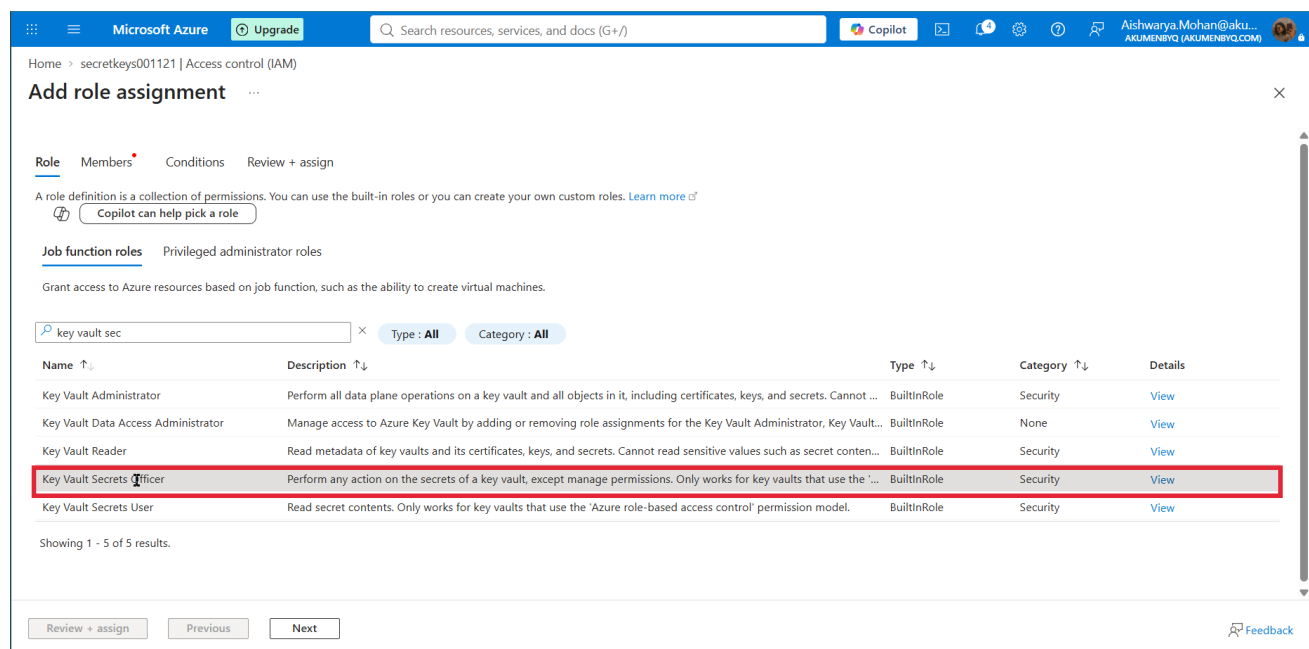
2.2 Opening Access control (IAM)

Access control is used to assign the permissions required to create and read secrets in the vault.



2.3 Granting permission to manage secrets

The Key Vault Secrets Officer role is selected so secrets can be created and managed in the vault.



Home > secretkeys001121 | Access control (IAM)

Add role assignment

Role Members Conditions Review + assign

A role definition is a collection of permissions. You can use the built-in roles or you can create your own custom roles. [Learn more](#)

[Copilot can help pick a role](#)

Job function roles Privileged administrator roles

Grant access to Azure resources based on job function, such as the ability to create virtual machines.

key vault sec Type: All Category: All

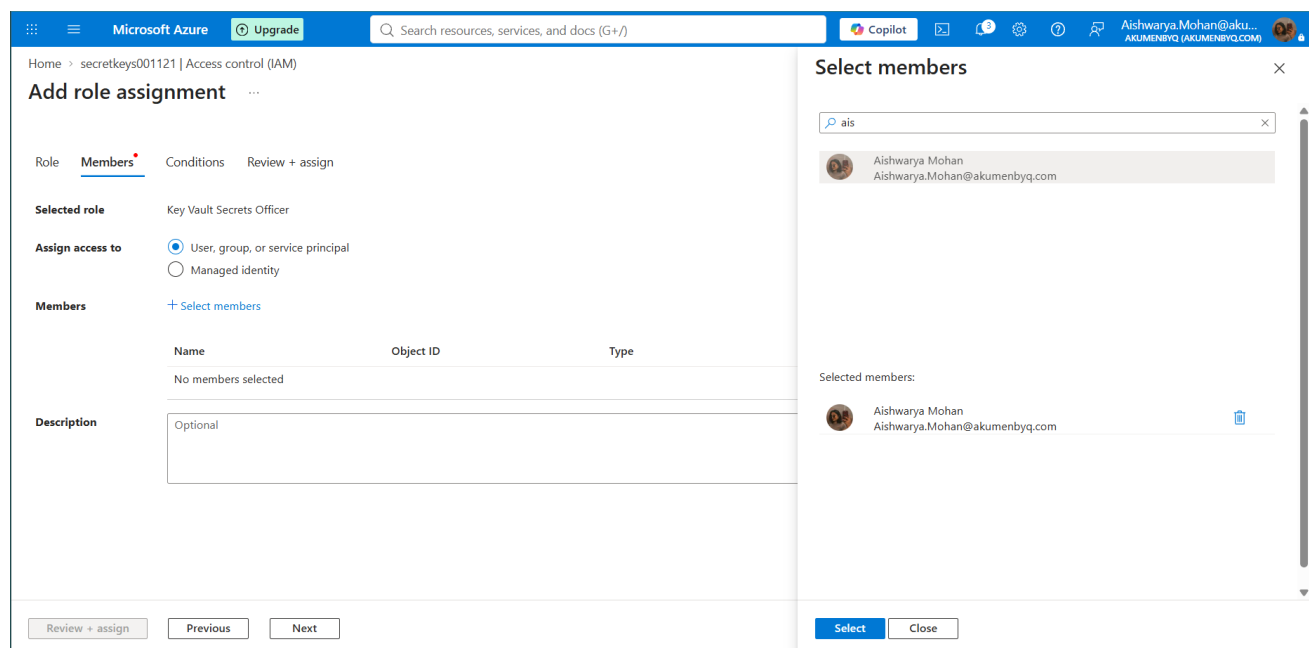
Name	Description	Type	Category	Details
Key Vault Administrator	Perform all data plane operations on a key vault and all objects in it, including certificates, keys, and secrets. Cannot ...	BuiltInRole	Security	View
Key Vault Data Access Administrator	Manage access to Azure Key Vault by adding or removing role assignments for the Key Vault Administrator, Key Vault...	BuiltInRole	None	View
Key Vault Reader	Read metadata of key vaults and its certificates, keys, and secrets. Cannot read sensitive values such as secret conten...	BuiltInRole	Security	View
Key Vault Secrets Officer	Perform any action on the secrets of a key vault, except manage permissions. Only works for key vaults that use the '...	BuiltInRole	Security	View
Key Vault Secrets User	Read secret contents. Only works for key vaults that use the 'Azure role-based access control' permission model.	BuiltInRole	Security	View

Showing 1 - 5 of 5 results.

[Review + assign](#) [Previous](#) [Next](#) [Feedback](#)

2.4 Selecting the member

The role is assigned to the required user account through the Members step.



Home > secretkeys001121 | Access control (IAM)

Add role assignment

Role **Members** Conditions Review + assign

Selected role Key Vault Secrets Officer

Assign access to ☒ User, group, or service principal ☐ Managed identity

Members [+ Select members](#)

Name	Object ID	Type
No members selected		

Description Optional

[Review + assign](#) [Previous](#) [Next](#)

Select members

ais

Aishwarya Mohan
Aishwarya.Mohan@akumenbyq.com

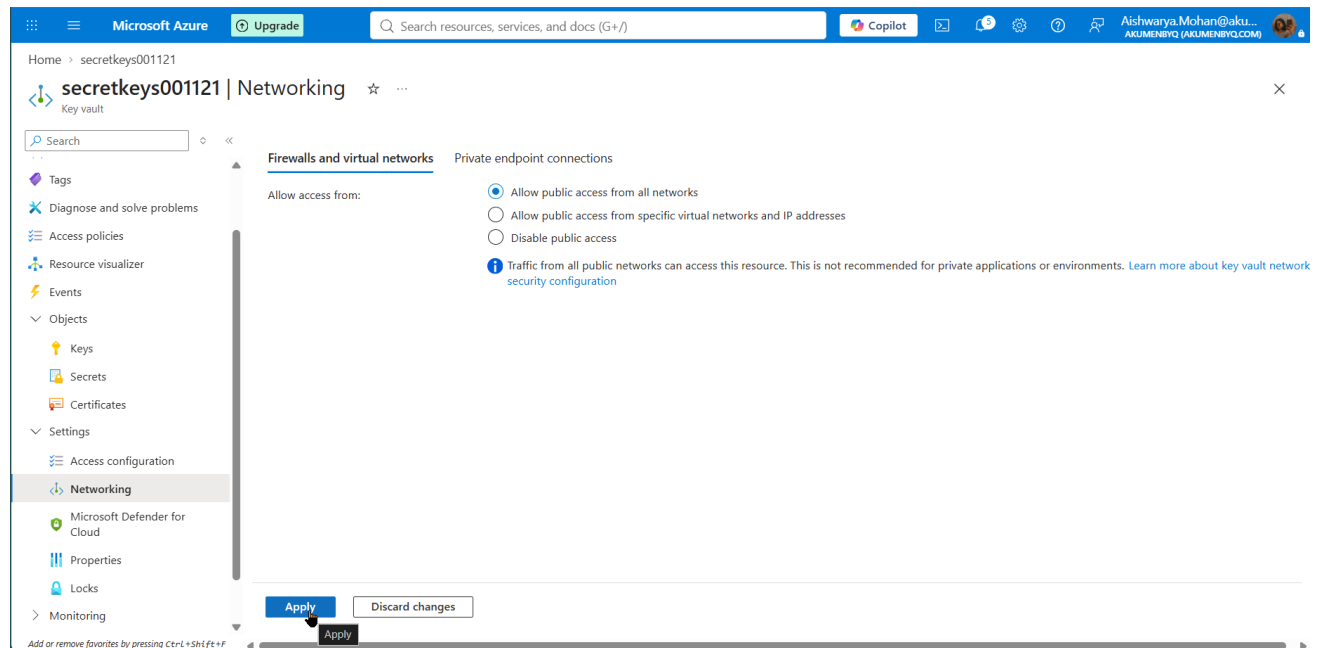
Selected members:

Aishwarya Mohan
Aishwarya.Mohan@akumenbyq.com

[Select](#) [Close](#)

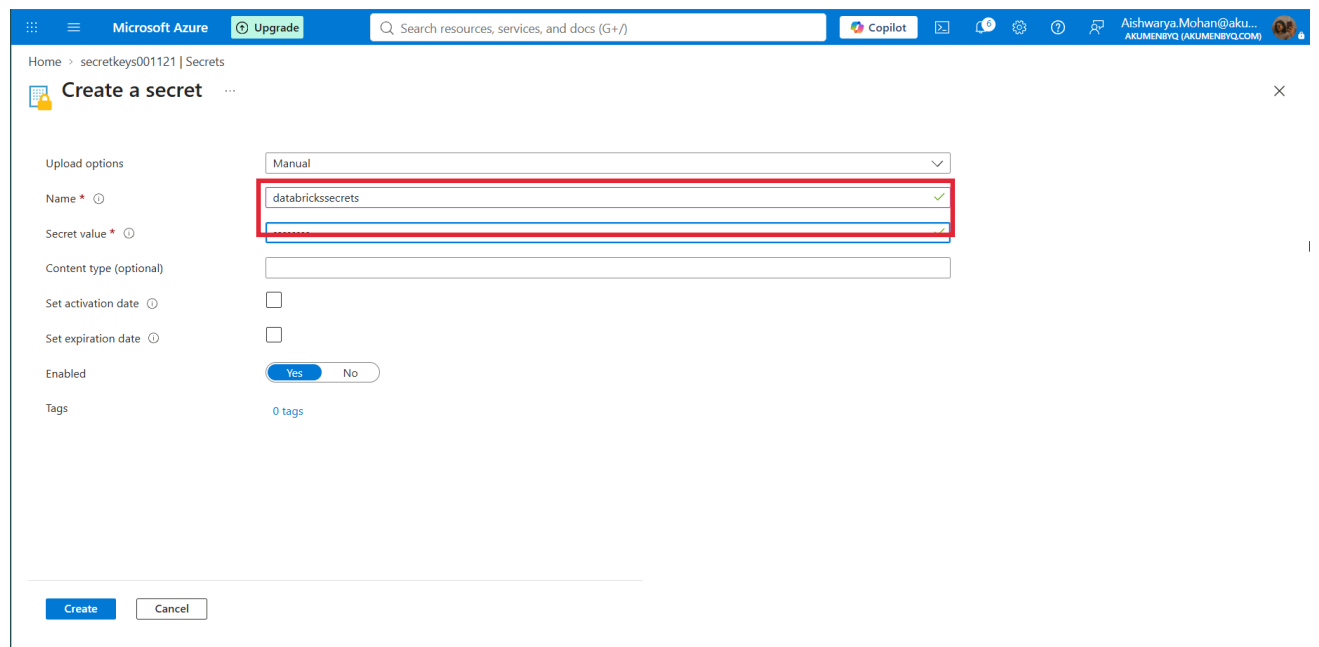
2.5 Allowing network access to the vault

For this test environment, public access from all networks is enabled so the Databricks workspace can reach the Key Vault endpoint.



2.6 Creating the first Key Vault secret

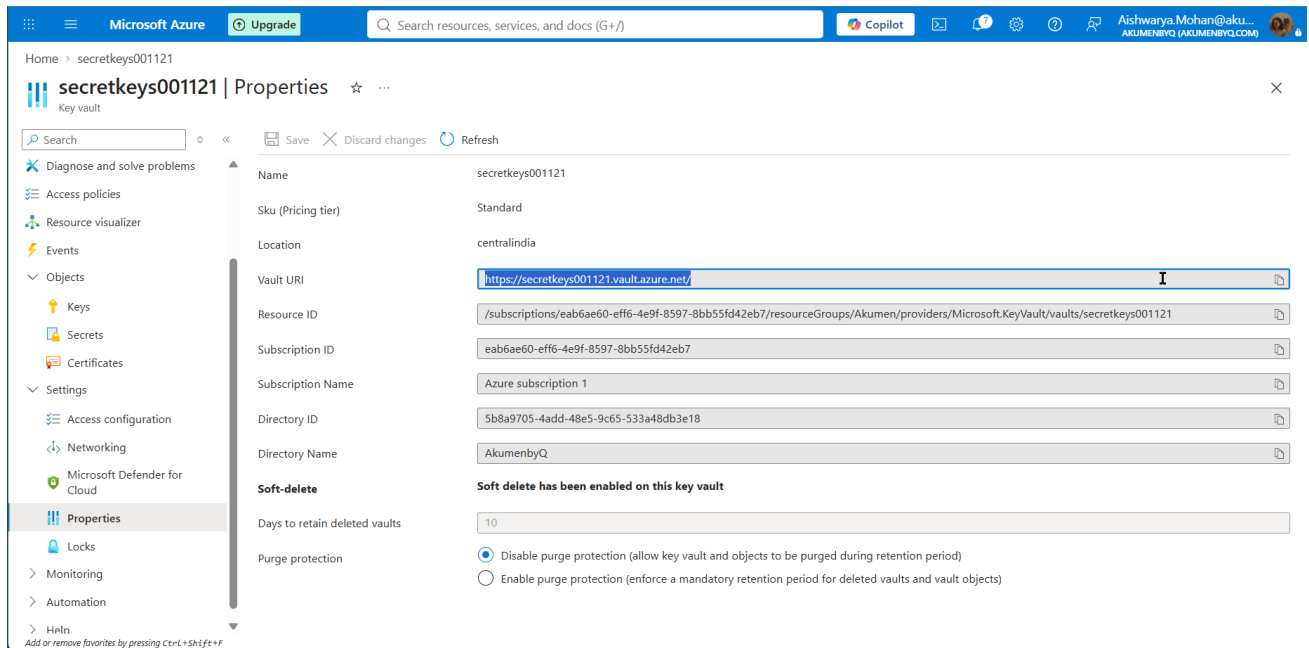
A secret named databrickssecrets is created in the vault. The actual secret value is not exposed in the screenshot.



3. Creating a Databricks secret scope backed by Key Vault

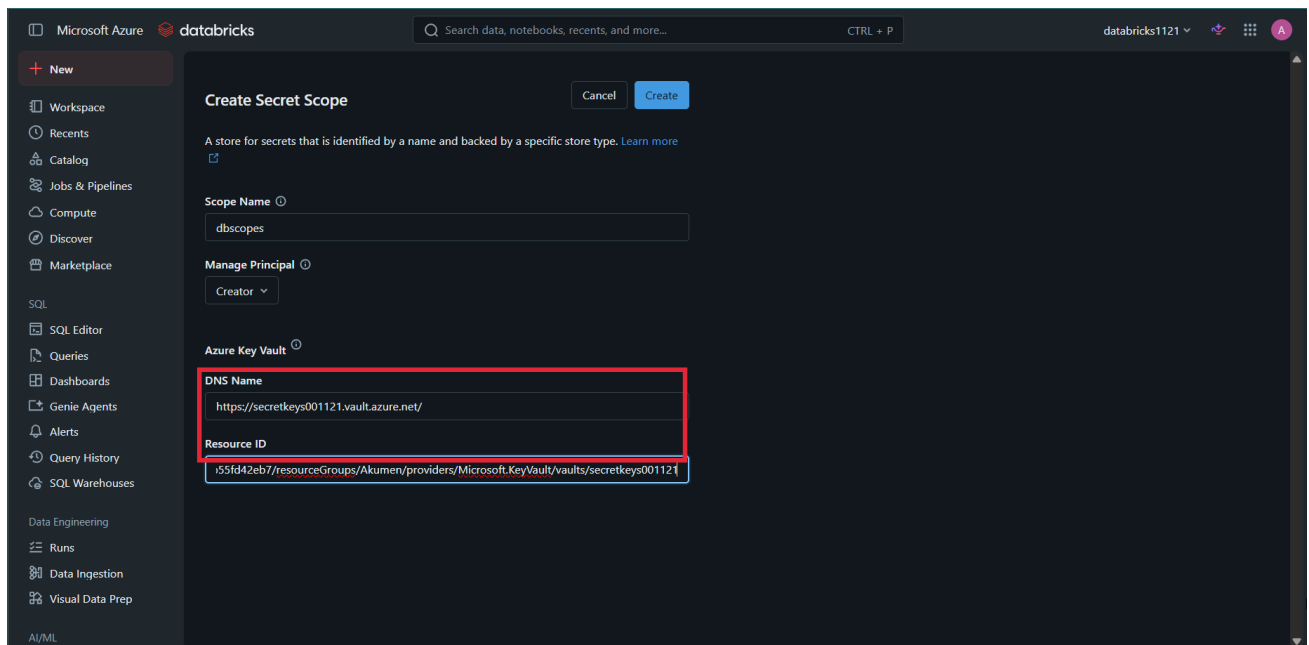
3.1 Getting the Key Vault values

The Vault URI and Resource ID are available on the Key Vault Properties page. These values are required when creating the Azure Key Vault-backed secret scope in Databricks.



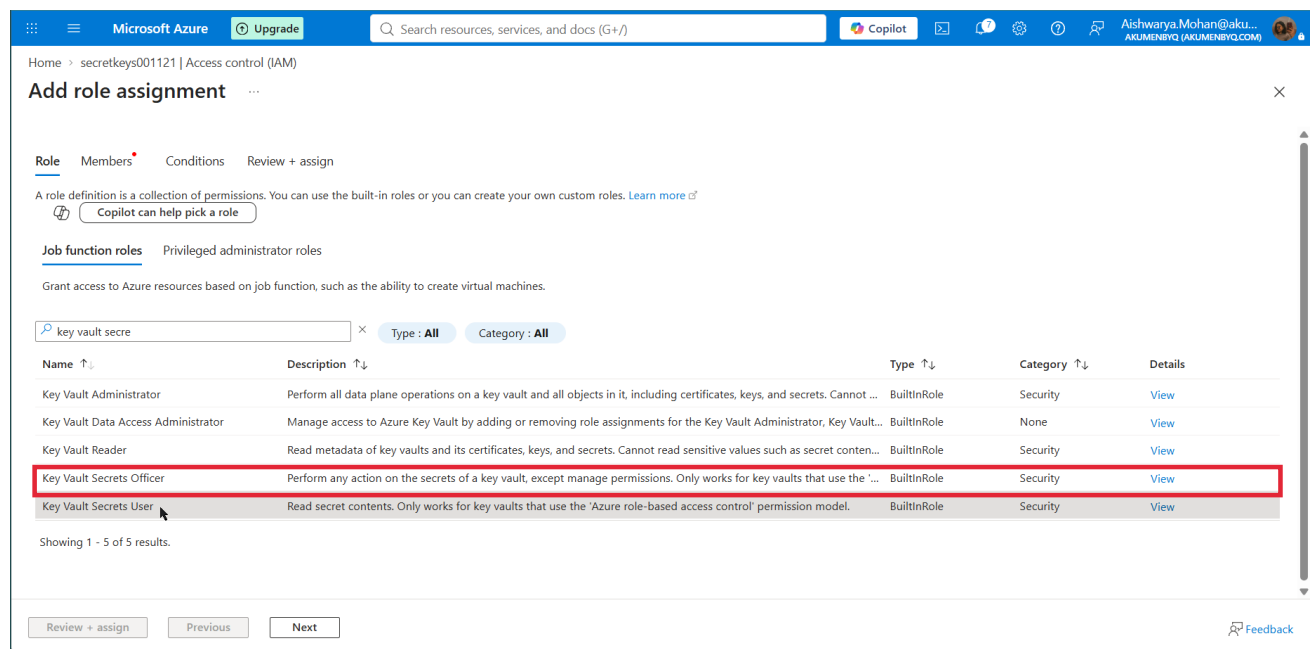
3.2 Creating the secret scope

A secret scope named dbscopes is created in Databricks and linked to the Azure Key Vault by using the vault DNS name and Resource ID.



3.3 Giving Databricks permission to read secrets

The Key Vault Secrets User role is selected so the Azure Databricks application can read secret contents from the vault.



Home > secretkeys001121 | Access control (IAM)

Add role assignment

Role Members Conditions Review + assign

A role definition is a collection of permissions. You can use the built-in roles or you can create your own custom roles. [Learn more](#) of Copilot can help pick a role

Job function roles Privileged administrator roles

Grant access to Azure resources based on job function, such as the ability to create virtual machines.

key vault secre Type: All Category: All

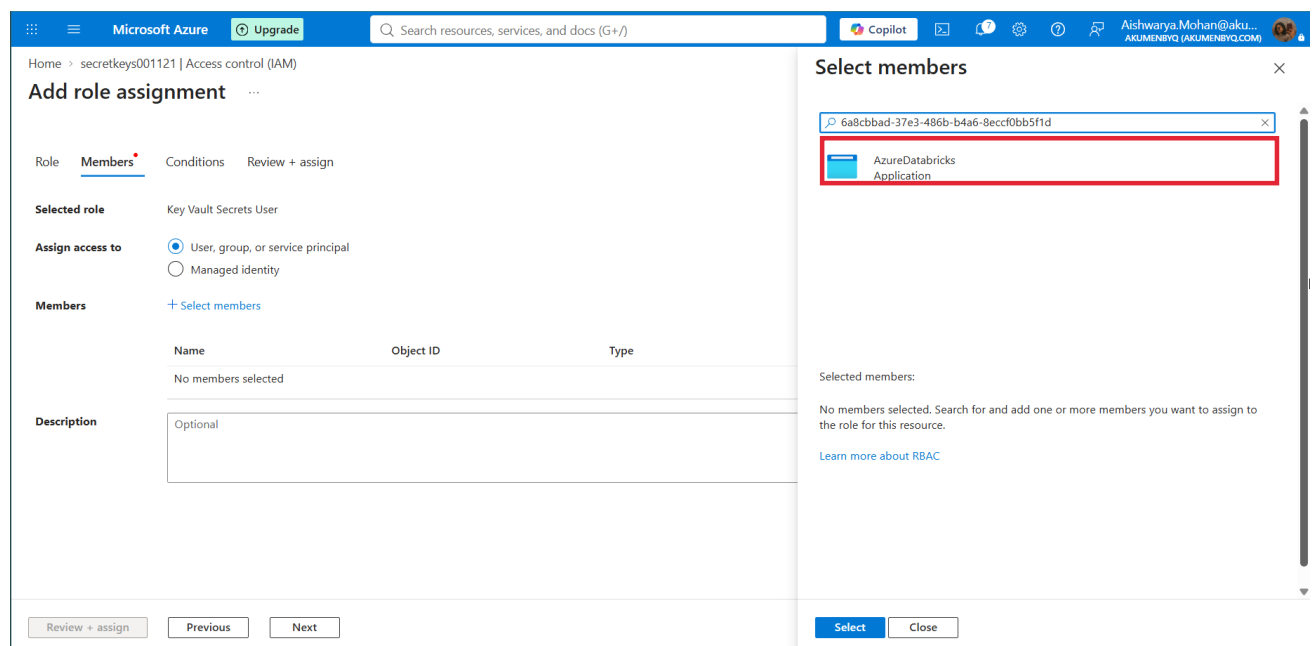
Name	Description	Type	Category	Details
Key Vault Administrator	Perform all data plane operations on a key vault and all objects in it, including certificates, keys, and secrets. Cannot ...	BuiltInRole	Security	View
Key Vault Data Access Administrator	Manage access to Azure Key Vault by adding or removing role assignments for the Key Vault Administrator, Key Vault...	BuiltInRole	None	View
Key Vault Reader	Read metadata of key vaults and its certificates, keys, and secrets. Cannot read sensitive values such as secret conten...	BuiltInRole	Security	View
Key Vault Secrets Officer	Perform any action on the secrets of a key vault, except manage permissions. Only works for key vaults that use the '...	BuiltInRole	Security	View
Key Vault Secrets User	Read secret contents. Only works for key vaults that use the 'Azure role-based access control' permission model.	BuiltInRole	Security	View

Showing 1 - 5 of 5 results.

Review + assign Previous Next

3.4 Selecting the Azure Databricks application

The AzureDatabricks application is selected as the member for the role assignment.



Home > secretkeys001121 | Access control (IAM)

Add role assignment

Role Members Conditions Review + assign

Selected role Key Vault Secrets User

Assign access to ☒ User, group, or service principal ☐ Managed identity

Members + Select members

Name	Object ID	Type
No members selected		

Description Optional

Review + assign Previous Next

Select members

6a8cbbad-37e3-486b-b4a6-8ecf0bb5f1d

AzureDatabricks Application

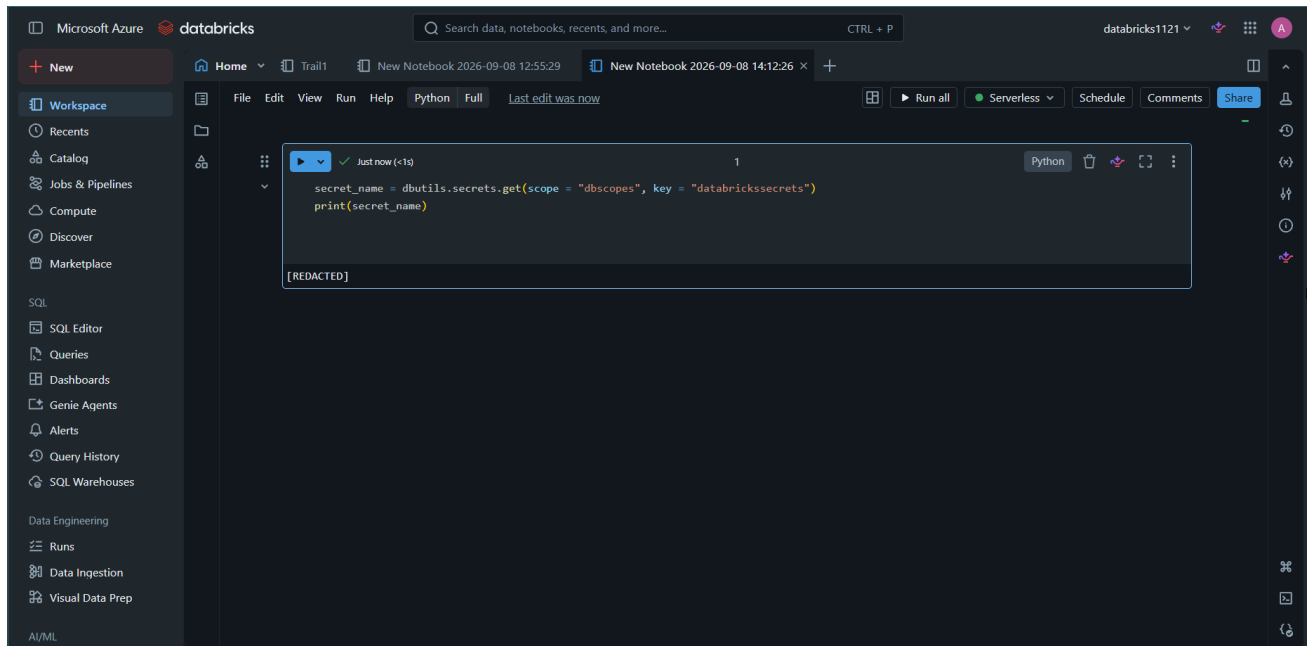
Selected members:

No members selected. Search for and add one or more members you want to assign to the role for this resource. [Learn more about RBAC](#)

Select Close

3.5 Retrieving the secret from Databricks

The `dbutils.secrets.get` command reads the Key Vault-backed secret through the `dbscopes` scope. Databricks automatically redacts the printed value, which confirms that the secret can be retrieved without displaying the credential.

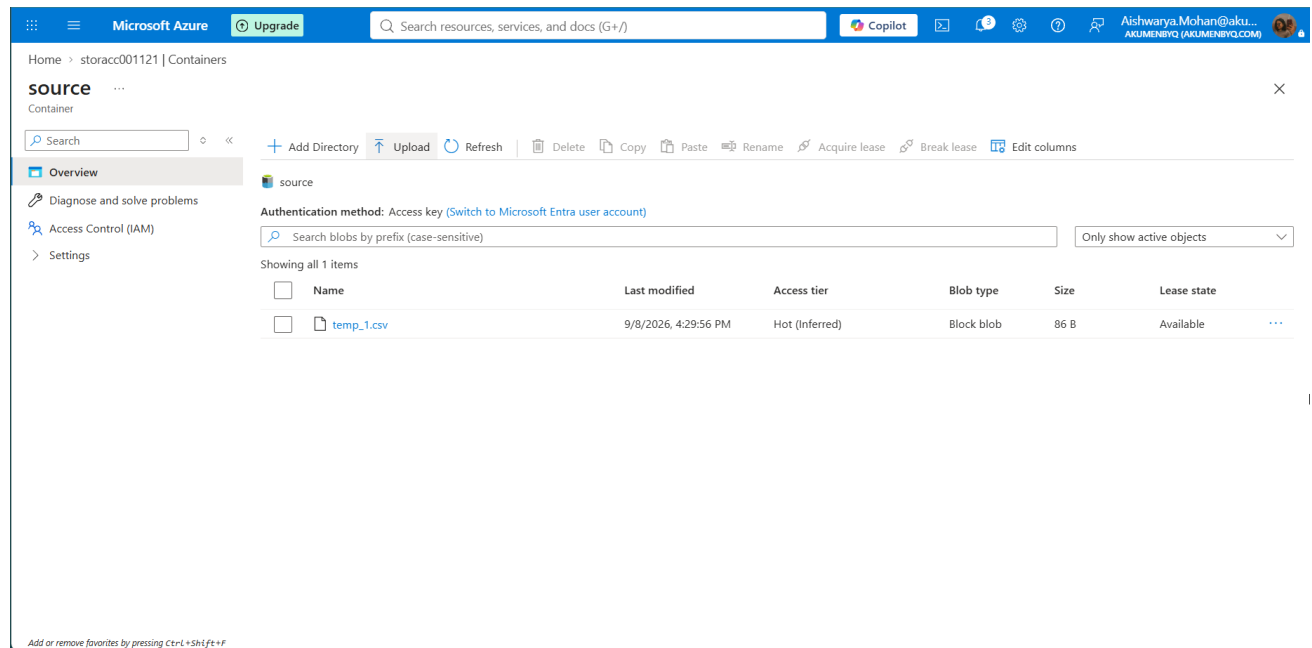


At this point the Key Vault integration is working: the secret scope resolves successfully and the secret value remains protected in notebook output.

4. Creating another storage secret and validating ADLS access

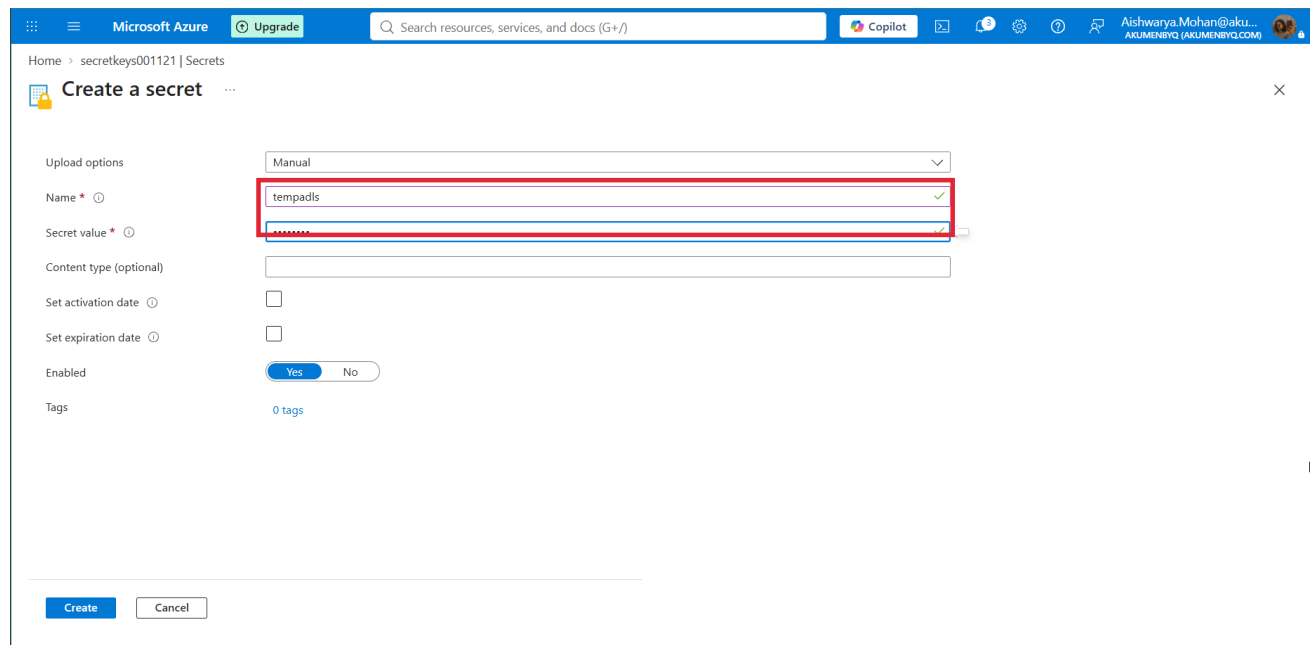
4.1 Uploading a second test file

A second CSV file, temp_1.csv, is placed in the source container for the final access check.



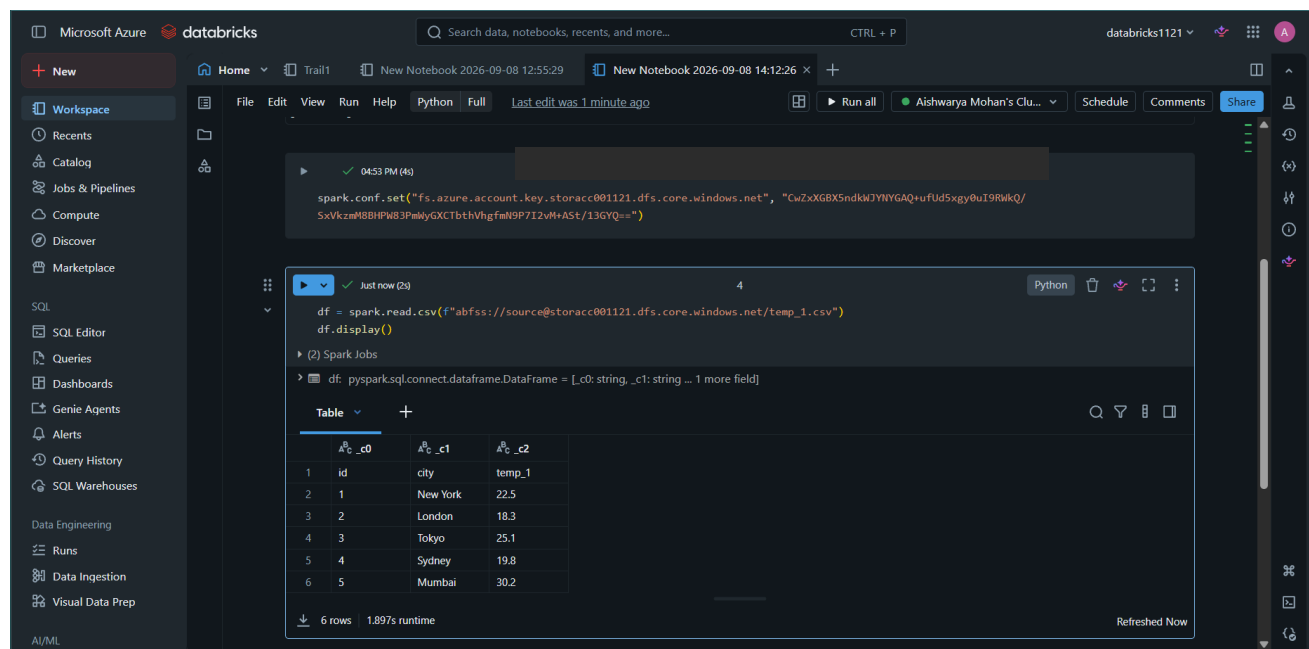
4.2 Storing the storage access key as another secret

A second Key Vault secret named tempadls is created to store the storage account access key. The secret value is hidden in the screenshot.



4.3 Reading the second file from ADLS Gen2

The storage access configuration is applied and temp_1.csv is read from the source container. The access key shown in the notebook is masked in this report.



The screenshot shows a Databricks notebook interface. The top menu bar includes 'File', 'Edit', 'View', 'Run', and 'Help'. The notebook is titled 'New Notebook 2026-09-08 14:12:26'. The code in the notebook is as follows:

```
spark.conf.set("fs.azure.account.key.storacc001121.dfs.core.windows.net", "CwZxXGBX5ndk7YNYGAQ+ufUd5xgy0u19RwkQ/SxVkcZmMB8HPw83PmMyGXCTbthVhgfmN9P7I2vM+AST/13GYQ==")
```

The output of the code is a table with 6 rows and 3 columns: id, city, and temp_1. The table is displayed below the code cell.

	id	city	temp_1
1	1	New York	22.5
2	2	London	18.3
3	3	Tokyo	25.1
4	4	Sydney	19.8
5	5	Mumbai	30.2

The table is displayed with 6 rows and 3 columns. The columns are labeled id, city, and temp_1. The data is as follows:

	id	city	temp_1
1	1	New York	22.5
2	2	London	18.3
3	3	Tokyo	25.1
4	4	Sydney	19.8
5	5	Mumbai	30.2

4.4 Final result

The file is displayed successfully in Databricks with the id, city and temp_1 columns. This confirms that the Databricks workspace can reach the ADLS Gen2 container and read the uploaded data.

Conclusion

The project first confirms direct ADLS Gen2 access from Databricks using a storage account key, then adds Azure Key Vault and a Databricks secret scope for protected secret retrieval. The final screenshots confirm that files stored in the source container can be read and displayed from the Databricks notebook.